

Автономная некоммерческая организация профессиональная образовательная
организация «УНИВЕРСИТЕТСКИЙ КОЛЛЕДЖ БРИКС»

Специальность: Информационные системы и программирование

ИНДИВИДУАЛЬНЫЙ ПРОЕКТ

на тему: Визуализация протокола WebSocket

Выполнил студент учебной группы:

ОИСП-591

Пак Александр Николаевич

Педагогический работник/Руководитель проекта:

Туев Юрий Юрьевич

(подпись) _____

Содержание

1. Введение.....	3-4
2. Глава 1. Теоретические основы протокола WebSocket.....	5-8
3. Глава 2. Практическая реализация визуализации WebSocket.....	9-17
4. Заключение.....	18-19
5. Список источников.....	20

Введение

В современном мире веб-разработки всё большее значение приобретают приложения реального времени. Онлайн-чаты, биржевые котировки, совместное редактирование документов, игровые платформы — все эти сервисы требуют мгновенной доставки данных от сервера к клиенту и обратно. Традиционный протокол HTTP, основанный на модели «запрос-ответ», для таких задач подходит плохо: клиент вынужден постоянно опрашивать сервер (polling), что создаёт избыточную нагрузку и задержки.

Решением этой проблемы стал протокол WebSocket, стандартизированный в 2011 году как RFC 6455. Он устанавливает постоянное двунаправленное соединение между клиентом и сервером, позволяя обмениваться данными в реальном времени с минимальными накладными расходами. Однако, несмотря на широкое распространение WebSocket, многие начинающие разработчики сталкиваются с трудностями при его изучении: процесс установки соединения, структура передаваемых фреймов, механизмы ping/pong и закрытия соединения остаются «за кадром» и не видны глазу.

Актуальность темы обусловлена несколькими факторами. Во-первых, язык Python широко используется как в технологических компаниях, так и в образовательных учреждениях. Он позволяет быстро переходить от простых примеров к более сложным проектам, включая графические и сетевые приложения. Во-вторых, понимание работы сетевых протоколов является обязательной частью подготовки специалиста в области информационных систем и программирования. В-третьих, визуализация сложных процессов делает обучение более наглядным и доступным, что особенно важно на начальных этапах изучения программирования.

Цель проекта — разработать программное приложение для визуализации работы протокола WebSocket, которое позволит в реальном времени наблюдать за процессом установки соединения, отправкой и получением сообщений, а также завершением сеанса связи.

Для достижения поставленной цели были определены следующие **задачи**:

1. Изучить теоретические основы протокола WebSocket, его отличия от HTTP и структуру данных.
2. Провести обзор существующих инструментов визуализации сетевых протоколов.
3. Определить требования к разрабатываемому приложению.
4. Выбрать оптимальный технологический стек для реализации.
5. Разработать архитектуру приложения, включающую серверную и клиентскую части.
6. Реализовать визуальное отображение состояния соединения, лога сообщений и передачи данных.
7. Провести тестирование приложения и проанализировать полученные результаты.

В ходе проекта была реализована рабочая версия приложения, которая включает WebSocket-сервер на Python и клиентское приложение с графическим интерфейсом на Tkinter. Приложение позволяет подключаться к серверу, отправлять и получать сообщения, а также визуально отслеживать все этапы работы протокола.

ГЛАВА I. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОТОКОЛА WEBSOCKET

Эволюция веб-коммуникаций: от HTTP к WebSocket

Для понимания значимости протокола WebSocket необходимо рассмотреть эволюцию веб-коммуникаций. Изначально Всемирная паутина строилась на основе протокола HTTP, который был разработан для передачи гипертекстовых документов. Ключевая особенность HTTP — это модель «запрос-ответ»: клиент инициирует запрос, сервер обрабатывает его и отправляет ответ, после чего соединение закрывается.

В традиционном вебе эта модель работала безупречно: пользователь открывал страницу, браузер отправлял запрос, сервер возвращал HTML - документ, и соединение завершалось. Однако с развитием веб-приложений возникла потребность в более динамичном обмене данными. Например, в онлайн-чатах, биржевых приложениях и многопользовательских играх информация должна поступать на экран пользователя мгновенно, без необходимости обновлять страницу.

Для решения этой проблемы были разработаны различные обходные пути:

1. Polling - клиент периодически отправляет запросы к серверу с вопросом «есть ли новые данные?». Сервер отвечает сразу или через некоторое время. Этот подход создает огромную нагрузку на сервер и сеть, особенно при большом количестве клиентов и высокой частоте запросов.
2. Long Polling - клиент отправляет запрос, а сервер удерживает его открытым до тех пор, пока не появятся новые данные или не истечет таймаут. После получения ответа клиент сразу отправляет следующий запрос. Этот метод снижает количество пустых запросов, но все еще требует постоянного переподключения.
3. Server-Sent Events - технология, позволяющая серверу отправлять данные клиенту без дополнительных запросов. Однако SSE работает только в одну сторону: от сервера к клиенту. Клиент не может отправлять данные обратно через это же соединение.

Все эти подходы были лишь временными решениями, так как они не обеспечивали полноценной двунаправленной связи и добавляли задержки.

Протокол WebSocket, стандартизированный IETF в 2011 году, стал ответом на эти проблемы. Он предоставляет возможность установить постоянное двунаправленное соединение между клиентом и сервером, по которому можно передавать данные в любой момент времени без необходимости создавать новое подключение. Это делает WebSocket идеальным выбором для приложений реального времени.

Жизненный цикл WebSocket-соединения

Процесс работы с протоколом WebSocket делится на три этапа: **установка соединения, обмен данными и закрытие.**

Установка соединения начинается с того, что клиент отправляет серверу HTTP GET-запрос со специальными заголовками Upgrade: websocket и Connection: Upgrade. В запросе также передается случайный ключ Sec-

WebSocket-Key и версия протокола. Сервер, поддерживающий WebSocket, отвечает кодом 101 Switching Protocols, подтверждая переход на новый протокол. В ответе он отправляет Sec-WebSocket-Асепт, который вычисляется на основе ключа клиента и фиксированного идентификатора. После проверки этого значения соединение считается установленным, и стороны переходят на WebSocket.

Обмен данными после установки соединения становится полноценно двунаправленным. Любая сторона может отправлять сообщения в любой момент. Данные передаются в виде фреймов, каждый из которых содержит заголовок с типом данных (текст, бинарные данные, управляющие сигналы) и полезную нагрузку. Управляющие фреймы ping и pong используются для проверки активности соединения. Все фреймы от клиента к серверу маскируются с помощью XOR-ключа для безопасности.

Заккрытие соединения инициируется любой стороной путем отправки close-фрейма с кодом причины. Получив его, вторая сторона обязана ответить аналогичным фреймом, после чего соединение разрывается. Если ответ не получен, соединение закрывается принудительно. При аварийном обрыве TCP-соединение закрывается по таймауту.

WebSocket-соединение всегда начинается с HTTP-запроса. Клиент отправляет на сервер обычный GET-запрос со специальными заголовками, которые указывают на желание «апгрейдить» соединение до WebSocket

Upgrade: websocket - указывает, что клиент хочет переключиться на протокол WebSocket.

Connection: Upgrade - подтверждает намерение изменить протокол.

Sec-WebSocket-Key - случайно сгенерированная строка, которую сервер использует для подтверждения соединения. Сервер должен добавить к этому ключу специальный GUID, вычислить SHA-1 хеш и вернуть в заголовке Sec-WebSocket-Accept.

После этого соединение считается установленным, и дальнейший обмен данными происходит уже по WebSocket-протоколу, который является более легковесным по сравнению с HTTP.

После успешного рукопожатия клиент и сервер могут обмениваться данными в виде **фреймов**. В отличие от HTTP, где каждое сообщение содержит полные заголовки, WebSocket-фреймы имеют минимальный размер, что позволяет эффективно передавать даже небольшие сообщения.

Сообщения могут быть:

Текстовые - содержат UTF-8 строки.

Бинарные - содержат произвольные двоичные данные.

Управляющие - ping, pong, close.

Любая из сторон может инициировать закрытие соединения, отправив специальный управляющий фрейм Close. В ответ другая сторона должна подтвердить получение этого фрейма, после чего соединение разрывается.

Структура WebSocket-фреймов

Понимание структуры фреймов важно для глубокого понимания протокола. Каждый WebSocket-фрейм начинается с одного из заголовков:

FIN - флаг, указывающий, является ли этот фрейм последним в сообщении.

Opcode - тип данных.

Mask - флаг маскировки данных.

Payload length - длина полезной нагрузки .

Обзор существующих инструментов визуализации

В настоящее время существует несколько инструментов, позволяющих анализировать WebSocket-трафик:

Wireshark - мощный сетевой анализатор, который может захватывать и отображать WebSocket-фреймы. Однако интерфейс Wireshark перегружен множеством низкоуровневых деталей, что делает его сложным для начинающих.

Chrome DevTools (Network Tab) - встроенный в браузер инструмент,

который может показывать WebSocket-фреймы. Однако информация представлена в виде простого списка без акцента на визуализацию этапов соединения.

WebSocket King - расширение для браузера, предназначенное для тестирования WebSocket-соединений, но без визуализации процесса.

Postman - поддерживает WebSocket, но ориентирован на тестирование, а не на образовательную визуализацию.

Недостаток всех этих инструментов - они либо слишком сложны для начинающих, либо не предоставляют наглядного представления о жизненном цикле соединения. Разрабатываемое приложение призвано заполнить этот пробел, предоставляя простой и понятный интерфейс для визуализации WebSocket в образовательных целях.

ГЛАВА II. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ВИЗУАЛИЗАЦИИ WEBSOCKET

Постановка задачи и требования к системе

Цель практической части — создать приложение, которое визуализирует работу WebSocket в реальном времени. Приложение должно быть простым в использовании и наглядно демонстрировать все этапы работы протокола.

Функциональные требования:

1. Подключение к WebSocket-серверу по указанному URL.
2. Отображение состояния соединения (Disconnected, Connecting, Connected, Closed).
3. Возможность отправки текстовых сообщений.
4. Отображение входящих сообщений с временной меткой.
5. Цветовое разделение входящих и исходящих сообщений.
6. Отображение системных событий (подключение, закрытие, ошибки).
7. Визуальное представление структуры фреймов.
8. Возможность закрытия соединения с двух сторон.

Нефункциональные требования:

1. Приложение должно быть кроссплатформенным.
2. Интерфейс должен быть интуитивно понятным.
3. Код должен быть структурирован и задокументирован.

Выбор технологического стека

Для реализации проекта был выбран следующий стек:

Python 3.14 - язык программирования, который обеспечивает простоту написания кода и наличие всех необходимых встроенных библиотек для работы с сетевыми протоколами.

Стандартная библиотека socket - используется для создания TCP-соединений. Данный подход позволяет наглядно продемонстрировать, как именно происходит установка соединения, отправка и получение данных. В отличие от использования готовых библиотек, работа с сокетами показывает студентам реальную структуру сетевых пакетов и процесс рукопожатия.

Библиотека `ssl` - используется для поддержки защищенных WebSocket-соединений, хотя в текущей реализации применяется незащищенный режим для упрощения отладки и наблюдения за передаваемыми данными.

Встроенные модули обработки данных: `base64`, `hashlib` - используются для генерации ключей и вычисления хешей в процессе рукопожатия. Эти модули демонстрируют криптографические основы протокола без использования сторонних криптографических библиотек.

Модуль `threading` — обеспечивает параллельную работу сервера и клиента в рамках одной программы. Это позволяет запускать тестовый сервер и демонстрационный клиент одновременно, не переключаясь между разными окнами терминала.

Visual Studio Code — среда разработки с поддержкой отладки Python и подсветкой синтаксиса

Отказ от использования высокоуровневых библиотек, таких как `websockets` или `aiows`, позволяет студентам увидеть внутреннее устройство протокола: как формируются фреймы, как происходит маскировка данных, как выглядит процесс рукопожатия на уровне сырых байтов. Такой подход формирует более глубокое понимание сетевых протоколов по сравнению с использованием готовых абстракций, скрывающих детали реализации. Кроме того, все используемые библиотеки входят в стандартную поставку Python, что исключает необходимость установки дополнительных пакетов и упрощает воспроизведение проекта на любом компьютере.

Архитектура приложения

Приложение построено по клиент-серверной архитектуре и включает два основных компонента, работающих в рамках одной программы. Такой подход был выбран для упрощения демонстрации и тестирования, поскольку не требуется запускать отдельно сервер и клиент — всё работает в одном процессе с использованием многопоточности.

Первый компонент — это локальный тестовый сервер, реализованный в классе `SimpleWebSocketServer`. Сервер создает TCP-сокеты, привязывается к

локальному адресу и порту 8765 и ожидает входящие подключения. При поступлении нового клиента сервер принимает соединение, выполняет рукопожатие по протоколу WebSocket и переходит в режим постоянного обмена сообщениями. Сервер работает в отдельном потоке, что позволяет ему функционировать параллельно с основным интерфейсом программы. Это решение обеспечивает возможность одновременной работы сервера и клиента без блокировки пользовательского интерфейса.

Второй компонент — это клиентская часть, реализованная в виде набора функций для подключения к серверу, выполнения рукопожатия, отправки и получения сообщений, а также корректного закрытия соединения. Клиент использует те же базовые механизмы работы с сокетами, что и сервер, но с добавлением маскировки данных, которая требуется по спецификации протокола для всех фреймов, отправляемых от клиента к серверу.

Взаимодействие между компонентами организовано через TCP-соединение. Клиент инициирует подключение, отправляя HTTP-запрос с заголовками Upgrade. Сервер обрабатывает запрос, извлекает ключ Sec-WebSocket-Key, вычисляет асепт-ключ и возвращает ответ с кодом 101 Switching Protocols. После этого соединение переключается на WebSocket-протокол, и стороны могут обмениваться сообщениями в виде фреймов. Каждое сообщение, отправленное клиентом, сервер обрабатывает и возвращает обратно в виде эхо-ответа, что наглядно демонстрирует двунаправленный характер протокола.

Управление программой осуществляется через консольное меню, которое предоставляет пользователю два основных режима работы: интерактивный режим и автоматическую демонстрацию. В интерактивном режиме пользователь может самостоятельно подключаться к серверу, отправлять сообщения и наблюдать за ответами. В режиме демонстрации программа автоматически выполняет последовательность действий, показывая все этапы работы протокола с паузами и пояснениями для каждого шага. Такая структура

делает приложение удобным как для самостоятельного изучения, так и для показа в учебных целях.

Реализация серверной части

Серверная часть реализована в классе SimpleWebSocketServer и отвечает за прием входящих WebSocket-соединений, обработку рукопожатия и эхо-ответ на полученные сообщения. Основная задача сервера — наглядно продемонстрировать работу протокола, поэтому он реализован максимально прозрачно, с подробным логированием каждого действия.

При создании экземпляра сервера в конструкторе `init` задаются параметры хоста и порта, а также инициализируются переменные для хранения серверного и клиентского сокетов. Метод `start` создает TCP-сокеты и запускает сервер в отдельном потоке.

Код запуска сервера:

```
def start(self):
    self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    self.server_socket.bind((self.host, self.port))
    self.server_socket.listen(1)
    self.server_socket.settimeout(1)
    self.is_running = True

    print(f"Тестовый сервер запущен на ws://{self.host}:{self.port}")

    server_thread = threading.Thread(target=self._run, daemon=True)
    server_thread.start()
```

Данный код демонстрирует создание сокета с параметрами `socket.AF_INET` и `socket.SOCK_STREAM`. Установка опции `SO_REUSEADDR` позволяет повторно использовать адрес после закрытия программы, что удобно при разработке. Метод `bind` привязывает сокет к локальному адресу и порту 8765, а `listen(1)` переводит сокет в режим ожидания подключений. Таймаут в одну секунду предотвращает бесконечное блокирование при ожидании соединения. Запуск сервера в отдельном потоке с параметром `daemon=True`

позволяет серверу автоматически завершаться при закрытии основной программы.

Метод `_run` реализует основной цикл сервера, который в бесконечном цикле принимает новые подключения. При успешном принятии соединения вызывается метод `_handle_client`, который обрабатывает все взаимодействие с клиентом.

Реализация клиентской части и визуализации

Клиентская часть реализована в виде набора функций, которые обеспечивают подключение к серверу, выполнение рукопожатия, отправку и получение сообщений, а также закрытие соединения. Визуализация работы протокола осуществляется через консольный вывод с подробными пояснениями на каждом этапе.

Генерация ключа для рукопожатия:

```
def generate_key():
    chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    rand_str = ""
    for _ in range(16):
        rand_str += random.choice(chars)
    return base64.b64encode(rand_str.encode()).decode()

def calculate_accept(ws_key):
    GUID = "258EAF55-E914-47DA-95CA-C5AB0DC85B11"
    combined = ws_key + GUID
    sha = hashlib.sha1(combined.encode()).digest()
    return base64.b64encode(sha).decode()
```

Эти функции демонстрируют криптографическую часть протокола. Функция `generate_key` создает случайную строку из 16 символов из набора `base64`. Функция `calculate_accept` реализует алгоритм вычисления ассерт-ключа, к полученному ключу добавляется фиксированный GUID, вычисляется SHA-1 хеш и результат кодируется в `base64`. Именно этот механизм обеспечивает защиту от случайных или злонамеренных попыток установить WebSocket-соединение без выполнения полного рукопожатия.

Корректное закрытие соединения:

```
def close_connection(sock):
    try:
        # Отправляем фрейм закрытия
        close_frame = bytes([0x88, 0x80, 0x00, 0x00, 0x00, 0x00])
        sock.send(close_frame)
        print("Отправлен фрейм закрытия")
        time.sleep(0.3)
    except:
        pass

    try:
        sock.close()
        print("Соединение закрыто")
    except:
        pass
```

Эта функция демонстрирует корректное закрытие WebSocket-соединения. Клиент формирует фрейм закрытия с кодом операции 0x88 (байт 0x88 означает текстовый фрейм закрытия с флагом FIN). Второй байт 0x80 указывает на замаскированный фрейм, затем следуют 4 байта маскирующего ключа. После отправки фрейма программа выдерживает паузу в 0.3 секунды, чтобы сервер успел обработать запрос, и затем закрывает TCP-сокеты.

Визуализация через консоль:

```
def print_line():
    print("=" * 60)

def print_header(text):
    print_line()
    print(f" {text}")
    print_line()

def wait_enter():
    input("\nНажми Enter чтобы продолжить:")

def log(message, emoji="•"):
    print(f"\n{emoji} {message}")
    time.sleep(0.3)
```

Эти вспомогательные функции создают визуальное оформление программы. Функция `print_line` выводит разделительную линию, `print_header` создает заголовки с рамками, `log` выводит сообщения с эмодзи и небольшой паузой для лучшего восприятия, а `wait_enter` приостанавливает выполнение до нажатия клавиши. В совокупности эти функции обеспечивают пошаговый режим демонстрации, позволяющий студентам внимательно изучить каждый этап работы протокола.

Тестирование и результаты

Тестирование разработанного приложения проводилось по нескольким сценариям, охватывающим все ключевые этапы работы WebSocket-протокола. Каждый сценарий был выполнен как в интерактивном режиме, так и в режиме автоматической демонстрации.

Успешное установление соединения

При запуске программы автоматически стартует тестовый сервер на порту 8765. При выборе интерактивного режима и вводе команды 2 происходит подключение к серверу. Клиент создает TCP-сокеты, устанавливает соединение с `localhost:8765` и выполняет рукопожатие. В процессе рукопожатия клиент генерирует случайный ключ `Sec-WebSocket-Key`, отправляет HTTP GET-запрос с заголовками `Upgrade` и получает ответ с кодом 101 `Switching Protocols`. На экран выводятся сообщения о каждом этапе: "Подключаюсь к `localhost:8765...`", "Соединение установлено", "WebSocket соединение установлено".

Отправка и получение сообщений

После установления соединения пользователь вводит команду 3 и текст сообщения. Функция `send_message` кодирует текст в WebSocket-фрейм с маскировкой и отправляет его серверу. На экране появляется сообщение "Отправлено: 'текст'". Сервер получает фрейм, расшифровывает его с помощью функции `_read_frame`, формирует ответное сообщение с префиксом "Эхо: " и отправляет обратно. Клиент принимает ответ, функция `receive_message`

выводит его на экран: "Получено: 'Эхо: текст'". Этот сценарий наглядно демонстрирует полный цикл двунаправленного обмена данными.

Обработка управляющих фреймов

При тестировании управляющих фреймов было проверено, что сервер корректно обрабатывает ping-фреймы (opcode 0x9), отвечая на них pong-фреймами. Также было проверено корректное закрытие соединения: при отправке close-фрейма (opcode 0x8) сервер отвечает подтверждением, и соединение разрывается. В логе появляются сообщения: "Отправлен фрейм закрытия", "Соединение закрыто".

Обработка ошибок

Был проведен тест на обработку ошибок. При попытке подключения к несуществующему серверу или при вводе некорректного адреса программа выводит сообщение об ошибке и предлагает повторить попытку. Также был протестирован сценарий разрыва соединения со стороны сервера: если сервер останавливается во время работы клиента, клиент обнаруживает разрыв соединения и выводит соответствующее сообщение.

Автоматическая демонстрация

В режиме автоматической демонстрации программа последовательно выполняет все шаги, описанные в сценариях выше, с паузами и пояснениями для каждого шага. Демонстрация включает шесть основных шагов: адрес сервера, создание сокета, подключение, рукопожатие, отправка трех тестовых сообщений и закрытие соединения. Каждый шаг сопровождается выводом пояснительного текста и ожиданием нажатия Enter от пользователя. Это позволяет использовать демонстрационный режим как учебное пособие, объясняющее работу протокола шаг за шагом.

Результаты тестирования

Все тестовые сценарии были выполнены успешно. Приложение стабильно работает на платформе Windows и должно корректно функционировать на других операционных системах, поддерживающих Python. Основные результаты тестирования:

Рукопожатие: клиент и сервер успешно выполняют HTTP-рукопожатие с генерацией и верификацией ключей. Все этапы рукопожатия логируются и отображаются на экране.

Формирование и парсинг фреймов: функции `encode_frame` и `decode_frame` корректно формируют и парсят WebSocket-фреймы в соответствии со спецификацией RFC 6455. Маскировка и демаскировка данных работают правильно.

Двунаправленный обмен: клиент и сервер успешно обмениваются сообщениями в обоих направлениях. Сервер корректно обрабатывает полученные сообщения и возвращает эхо-ответы.

Управляющие фреймы: сервер корректно обрабатывает `ping`-фреймы, отвечая `pong`-фреймами, и правильно реагирует на `close`-фреймы.

Обработка ошибок: программа корректно обрабатывает ошибки подключения, разрывы соединения и другие исключительные ситуации, выводя понятные сообщения об ошибках.

Визуализация: консольный интерфейс с разделительными линиями, эмодзи и пошаговым режимом обеспечивает наглядное отображение всех этапов работы протокола. Пошаговый режим с ожиданием нажатия `Enter` делает процесс обучения интерактивным и удобным для студентов.

Таким образом, разработанное приложение успешно демонстрирует все основные механизмы протокола WebSocket и может быть использовано как учебный материал для студентов, изучающих сетевые протоколы и программирование на Python

ЗАКЛЮЧЕНИЕ

В ходе выполнения индивидуального учебного проекта была разработана программа визуализации протокола WebSocket на языке Python. В ходе работы были решены следующие задачи:

Изучены теоретические основы протокола WebSocket, его отличия от HTTP, жизненный цикл соединения, структура фреймов и механизм рукопожатия. Это позволило глубоко понять принципы работы протокола и реализовать его на низком уровне с использованием стандартных библиотек Python.

Проведен обзор существующих инструментов визуализации, таких как Wireshark, Chrome DevTools и другие, выявлены их недостатки для образовательных целей и обоснована необходимость разработки собственного инструмента.

Определены требования к разрабатываемому приложению, выбраны технологии реализации и разработана архитектура приложения, включающая локальный тестовый сервер, клиентскую часть и консольный интерфейс управления.

Реализован полноценный WebSocket-сервер на стандартной библиотеке socket, поддерживающий рукопожатие, прием и отправку сообщений, обработку управляющих фреймов и корректное закрытие соединения. Код сервера включает методы чтения и парсинга фреймов, а также отправки эхо-ответов.

Реализован WebSocket-клиент с функциями подключения, отправки и получения сообщений, а также корректного закрытия соединения с использованием механизма маскировки данных. Клиентские функции демонстрируют формирование замаскированных фреймов и обработку ответов сервера.

Разработана система визуализации в виде консольного интерфейса с разделительными линиями, эмодзи, пошаговым режимом и подробным

логированием всех этапов работы протокола. Функции `print_line`, `print_header`, `log` и `wait_enter` обеспечивают наглядное представление процесса.

Проведено тестирование приложения по всем ключевым сценариям, подтвердившее корректность реализации и стабильность работы. Протестированы успешное подключение, отправка и получение сообщений, обработка управляющих фреймов и закрытие соединения.

Создание инструмента визуализации WebSocket показывает, что даже проект, выполненный на стандартных библиотеках Python без использования сторонних фреймворков, может стать эффективным учебным пособием для изучения сетевых протоколов. В моём случае проект стал не только попыткой воспроизвести работу популярного протокола, но и средством для закрепления навыков работы с сетевыми сокетами, битовыми операциями и проектирования клиент-серверных приложений.

В заключение, выполнение данного проекта позволяет создать полноценное учебное приложение и дает возможность практиковаться с реальными сетевыми протоколами. Для студента-программиста такая работа является важным шагом к пониманию низкоуровневых механизмов работы сети и подготовкой к профессиональной деятельности в области разработки сетевых приложений и систем реального времени. Исходный код проекта размещен на платформе GitHub, что позволяет использовать его как пример для дальнейшего улучшения и усовершенствования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. [RFC 6455 — The WebSocket Protocol. IETF.](#)
2. [АЛГОРИТМИЗАЦИЯ И ПРОГРАММИРОВАНИЕ. Учебник для СПО \(Якимов С. П.\)](#)
3. [ОСНОВЫ АЛГОРИТМИЗАЦИИ И ПРОГРАММИРОВАНИЯ 4-е изд. Учебник для СПО \(Трофимов В. В., Павловская Т. А.\)](#)
4. [ПРОГРАММИРОВАНИЕ НА PYTHON 6-е изд., пер. и доп. Учебник для СПО \(Федоров Д. Ю.\)](#)
5. [Документация Python. Модуль socket.](#)
6. [Документация Python. Модуль threading.](#)
7. [Документация Python. Модуль hashlib.](#)
8. [Документация Python. Модуль base64.](#)
9. [MDN Web Docs. WebSocket API.](#)
10. [GitHub с приложением.](#)